

	Carátula para entrega de prácticas	
Facultad de Ingeniería	Laboratorios de docencia	

Laboratorio de Computación

Salas A y B

Profesor(a): Alan Hernandez Fontes

Asignatura: Modelos de Programación Orientada a Objetos

Grupo: 07

No de Práctica(s): 4

Integrante(s): Torres Sanchez Efren

Vega Diaz Carlos Gael

No. de lista o brigada: 13 ,14

Semestre: 2027-1

Fecha de entrega: 27/09/2026

Observaciones:

CALIFICACIÓN:

OBJETIVOS:

Utilizar variables y arreglos con tipos de datos primitivos y con bibliotecas propias del lenguaje.

Ejercicio 1

Descripción del problema:

Leer 10 números y mostrar el mayor y el menor.

Requerimientos

- Leer 10 números
- Mostrar mayor y menor

Solución:

Para encontrar el número más grande y el más pequeño de un grupo de 10 datos, el programa primero le pide al usuario que ingrese los 10 números uno por uno y los guarda en una lista. Luego, toma el primer número como referencia inicial para el mayor y el menor, y empieza a compararlo con el resto de los elementos. Cada vez que encuentra un número más grande, actualiza la variable del mayor, y si encuentra uno más pequeño, actualiza la del menor. Al terminar de revisar todos los datos, muestra ambos resultados en pantalla.

Implementación

Utiliza las siguientes estructuras y componentes clave:

- **Arreglo estático:** Reservado con un tamaño fijo de 10 posiciones para almacenar los números decimales ingresados.
- **Captura de datos:** Utiliza un bucle for tradicional indexado desde 0 hasta 9 (`numeros.length`) para solicitar secuencialmente cada número mediante el método `nextDouble()`.

- **Procesamiento mediante *for-each*:**
 - Se utiliza un primer ciclo for (double numero : numeros) que compara iterativamente los elementos contra la variable max mediante una condición if.
 - Se implementa un segundo ciclo *for-each* análogo para determinar el valor asignado a la variable min.
- **Salida** Imprime los resultados mediante System.out.println() desplegando claramente ambos límites hallados.

Evidencia del Funcionamiento

```
Ingresa el número 1: 5
Ingresa el número 2: 6
Ingresa el número 3: 7
Ingresa el número 4: 56
Ingresa el número 5: 7
Ingresa el número 6: 5
Ingresa el número 7: 1
Ingresa el número 8: 9
Ingresa el número 9: 10
Ingresa el número 10: 3
El número mayor es: 56.0
El número menor es: 1.0
```

Ejercicio 2

Descripción del problema

Administrar un arreglo de 20 calificaciones mostrando promedio, máximo, mínimo y alumnos aprobados.

Requerimientos

- 20 calificaciones en un arreglo
- Mostrar promedio
- Mostrar maximo

- Mostar mínimo y alumnos aprobados

Solución

Utilizar un arreglo para almacenar las 20 calificaciones y una clase que se encarga de procesar estos datos. A partir del arreglo se realizan las operaciones necesarias para obtener el promedio, la calificación máxima y mínima, así como la cantidad de alumnos aprobados.

Implementación

La solución se realizó mediante la clase Calificaciones, que recibe el arreglo de calificaciones a través de su constructor. El atributo calificaciones se declaró como un arreglo de tipo double, permitiendo almacenar las 20 calificaciones del grupo.

Para calcular el promedio se utilizó el método calcularPromedio(), donde un ciclo **for** recorre cada elemento del arreglo y acumula los valores en la variable suma. Después, la suma se divide entre calificaciones.length, obteniendo así el promedio sin tener que establecer manualmente la cantidad de elementos.

Para obtener la calificación máxima y mínima se implementaron los métodos obtenerMaximo() y obtenerMinimo(). Ambos comienzan tomando el primer elemento del arreglo como referencia y posteriormente utilizan un ciclo **for** junto con una condición **if** para comparar cada calificación y **actualizar** el valor cuando se **encuentra una mayor o menor**.

Para determinar los alumnos aprobados se utilizó el método contarAprobados(). En este caso, se recorre el arreglo y mediante **if** (calificacion >= 70) se identifica si cada alumno cumple con la calificación mínima aprobatoria. Por cada resultado que cumple la condición, se incrementa el contador aprobados.

Evidencia de funcionamiento

```
Calificaciones analizadas:
```

```
Alumno 1: 85.0  
Alumno 2: 72.0  
Alumno 3: 90.0  
Alumno 4: 65.0  
Alumno 5: 78.0  
Alumno 6: 55.0  
Alumno 7: 88.0  
Alumno 8: 92.0  
Alumno 9: 70.0  
Alumno 10: 64.0  
Alumno 11: 76.0  
Alumno 12: 81.0  
Alumno 13: 59.0  
Alumno 14: 95.0  
Alumno 15: 73.0  
Alumno 16: 68.0  
Alumno 17: 87.0  
Alumno 18: 71.0  
Alumno 19: 60.0  
Alumno 20: 84.0
```

```
===== RESULTADOS =====
```

```
Promedio: 75.65  
Calificación máxima: 95.0  
Calificación mínima: 55.0  
Alumnos aprobados: 14
```

Ejercicio 3.

Descripción del problema

Simular un inventario con arreglos paralelos (nombre, precio y existencia), permitiendo búsquedas, actualización y cálculo del valor total del inventario.

Requerimientos:

- Utilizar arreglos paralelos
- Hacer búsqueda
- Hacer actualización

- Calcular el valor total del inventario

Solución:

Hacer un arreglo dinámico haciendo uso de ArrayList, de clases objeto previamente definida, dándole el método para poder hacer una búsqueda, el método para calcular el valor total del inventario y finalmente el método para Actualizar objetos dentro del inventario

Implementación:

La solución se implementó mediante tres clases:

- **Clase Inventario:** Contiene la lógica principal del sistema sobre una lista dinámica ArrayList<Objeto>. Sus métodos clave son:
 - **InsertarObjeto:** Crea y añade una nueva instancia de Objeto a la lista.
 - **Busqueda:** Recorre el arreglo mediante un bucle for y compara cadenas para retornar la referencia del artículo encontrado o null.
 - **Actualizar:** Utiliza Busqueda para localizar el elemento y modificar sus atributos de precio y existencia.
 - **ValorTotal:** Calcula el valor acumulado del inventario multiplicando precio por existencia de cada objeto registrado.
 - **tamañoAct:** Retorna la cantidad actual de elementos en la lista.
- **Clase Objeto:** Define la estructura del modelo de datos (nombre, precio, existencia).
- **Clase Menu:** Maneja la interfaz de consola mediante un ciclo do-while y un Scanner para capturar la entrada del usuario y ejecutar las operaciones descritas en Inventario.

Pruebas y resultados:

Si el inventario está vacío y se intenta buscar un objeto se avisa que el inventario aún está vacío:

```
¿Qué deseas hacer?  
1.Agregar un objeto al inventario  
2.Buscar un objeto en el inventario  
3.Actualizar  
4.Valor total  
5.Salir  
Respuesta:2  
  
El inventario aun esta vacio
```

Permite ingresar objetos al inventario:

```
¿Qué deseas hacer?  
1.Agregar un objeto al inventario  
2.Buscar un objeto en el inventario  
3.Actualizar  
4.Valor total  
5.Salir  
Respuesta:1  
  
Nombre: Goyo dorado  
Precio: 1000  
Existencia: 5  
  
¿Qué deseas hacer?  
1.Agregar un objeto al inventario  
2.Buscar un objeto en el inventario  
3.Actualizar  
4.Valor total  
5.Salir  
Respuesta:1  
  
Nombre: Xbox  
Precio: 10000  
Existencia: 3
```

Ahora que hay objetos dentro del inventario si se puede hacer la búsqueda e informa si está o no dentro del inventario:

```
¿Qué deseas hacer?  
1.Agregar un objeto al inventario  
2.Buscar un objeto en el inventario  
3.Actualizar  
4.Valor total  
5.Salir  
Respuesta:2  
  
Nombre: Goyo dorado  
  
Encontramos el objeto:  
Nombre: Goyo dorado  
Precio: 1000.00  
Existencia: 5
```

Se puede actualizar la información de cualquiera de los objetos:

¿Qué deseas hacer?

- 1.Agregar un objeto al inventario
- 2.Buscar un objeto en el inventario
- 3.Actualizar
- 4.Valor total
- 5.Salir

Respuesta:3

Que articulo deseas actualizar?

Xbox

Cual es el nuevo precio?

15000

Cuantas unidades hay?

2

Tu articulo fue actualizado

¿Qué deseas hacer?

- 1.Agregar un objeto al inventario
- 2.Buscar un objeto en el inventario
- 3.Actualizar
- 4.Valor total
- 5.Salir

Respuesta:2

Nombre: Xbox

Encontramos el objeto:

Nombre: Xbox

Precio: 15000.00

Existencia: 2

Calcula el valor total:

```
¿Qué deseas hacer?  
1.Agregar un objeto al inventario  
2.Buscar un objeto en el inventario  
3.Actualizar  
4.Valor total  
5.Salir  
Respuesta:4  
  
El valor total del inventario es: 35000.0
```

Conclusión

Durante el desarrollo de esta práctica logramos aplicar los conceptos de la programación orientada a objetos en Java mediante arreglos tradicionales y estructuras dinámicas. Aprendimos un poco sobre como estructurar información de forma eficiente, desde el uso de ciclos para identificar valores máximos y mínimos hasta la organización del código en clases y métodos independientes para calcular promedios y controlar calificaciones. Además, la implementación del inventario nos permitió comprender las ventajas de las colecciones dinámicas y la división en múltiples clases para modelar escenarios reales de búsqueda y actualización.

Bibliografías

- Facultad de Ingeniería, Universidad Nacional Autónoma de México. (2018). Manual de prácticas del Laboratorio de Modelos de Programación Orientada a Objetos (Código MAD0-21, versión 02).
- Google. (2026). *Gemini* (Versión de 2026) [Modelo de lenguaje de inteligencia artificial]. <https://gemini.google.com>